Assignment 3 Report

Software Development for Robotics

Group: PPD2

V. Malaxa s2726254, R. Díaz s3681548

March 4, 2026

3.1 - Linked List

The main objective of this subassignment was to understand the working principles of lists in C++, by creating some empty char strings, filling them with the elements of a string and then concatenating different lists.
(a) For this step, the provided header file List.h was included and initialized two empty char lists.
(b) The function fillList()[1] was created and consists of a void function that takes a string and a char list as inputs, in order to fill the char list with the characters of the string. It's worth noting that both the string and the list are passed by reference, this being especially important in the case of the list, where not including this "&" would lead to the list being copied into the function, with the original list in main() remaining empty.

Listing 1: fillList function

```
void fillList(const std::string& str, List<char>& list) {       //Fill
    list by inserting characters of string from the back.
    for (char c : str) {
        list.insertAtBack(c);
    }
}
```

(c) The two previously initialized empty lists were filled with the contents of the specified strings by using the created function **fillList()**. In order to print the two lists after being filled, the **print()** function present in the provided List.h header file was used.
(d) The provided header file List.h was edited by adding the concatenate function[2]. The function first checks whether the list that will not be the base is empty. If this is the case, the function does nothing. After checking this, in the case that the base list is the one that is empty, the function will take over the base list with the other concatenated function. Finally, if both lists have contents, the last component of the base list is connected to the first component of the other concatenated list, and the last component of the updated list is updated to be the first component of the other concatenated list. After all of this has been done, the list that was used for concatenation and was not the base is wiped, clearing all of its contents.

Listing 2: concatenate function

```
void concatenate(List<NODETYPE>& otherList) {
      if (otherList.isEmpty()) {
        return; // do nothing (empty)
    }

    if (this->isEmpty()) {
        // if current list empty, take over other list
        firstPtr = otherList.firstPtr;
        lastPtr = otherList.lastPtr;
    } else {
        // connect last of this list to first of the new one
        lastPtr->nextPtr = otherList.firstPtr;
        lastPtr = otherList.lastPtr;
    }

    // empty the other list
    otherList.firstPtr = nullptr;
    otherList.lastPtr = nullptr;
   }
```

(e) The concatenate function was tested by initializing and filling two additional lists and using the created function to concatenate both of them to one of the two other lists (list2). After printing, the results were as expected, as list2 showed its original contents, followed by the contents of the newly created lists in the expected order.

Testing for edgecases

The first test performed consisted on filling all the lists with empty strings. The result is the instant deletion of all of the lists and can be seen in Figure 1.

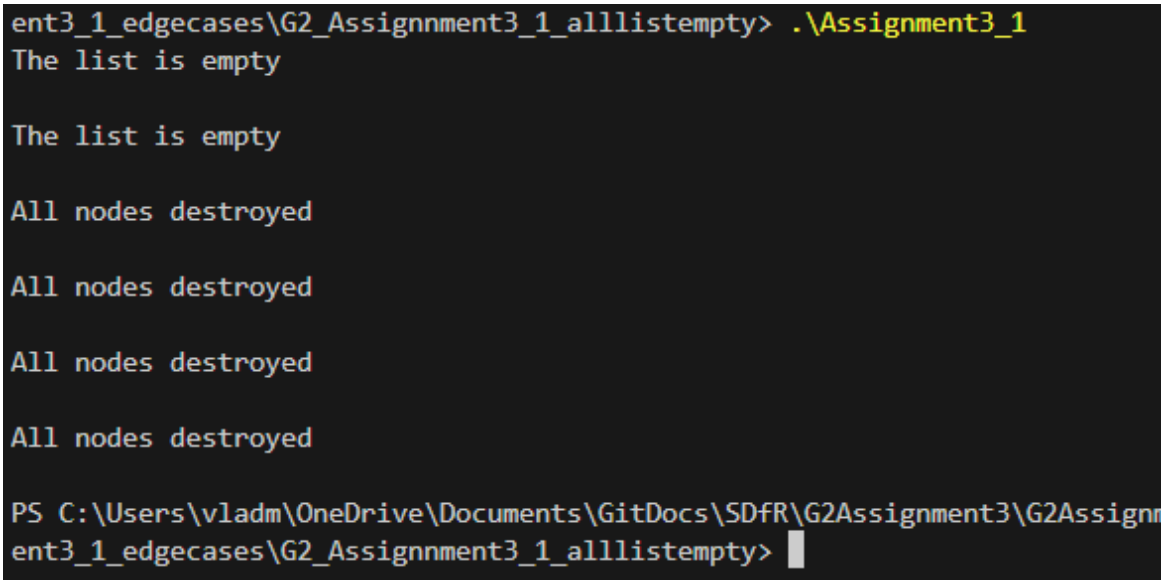


Figure 1: Results of testing the code with empty lists

The second test performed verified if the lists are empty after concatenation. The result can be seen in Figure 2. The first 3 lists were printed, with the first two showing up as intended, and in place of the third the message: "All nodes destroyed". This confirms the third list is empty.

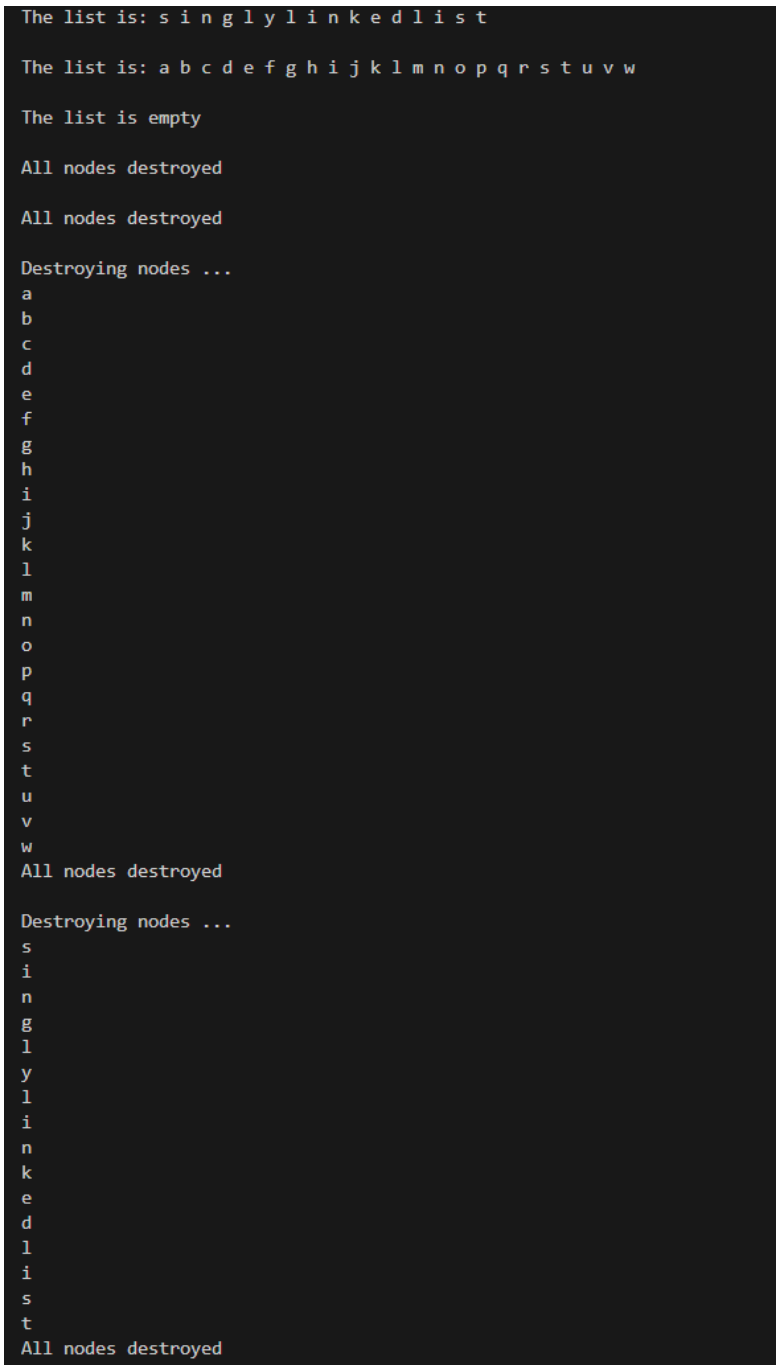


Figure 2: Results of concatenation on the list.

For the third test the second list was concatenated with itself. The result in Figure 3 shows that before printing of the second list, it has been wiped. Concatenation deletes the list used after adding the string to the desired location. If it is used on itself, it will result in the emptying of list2, as list2 is also the used list.

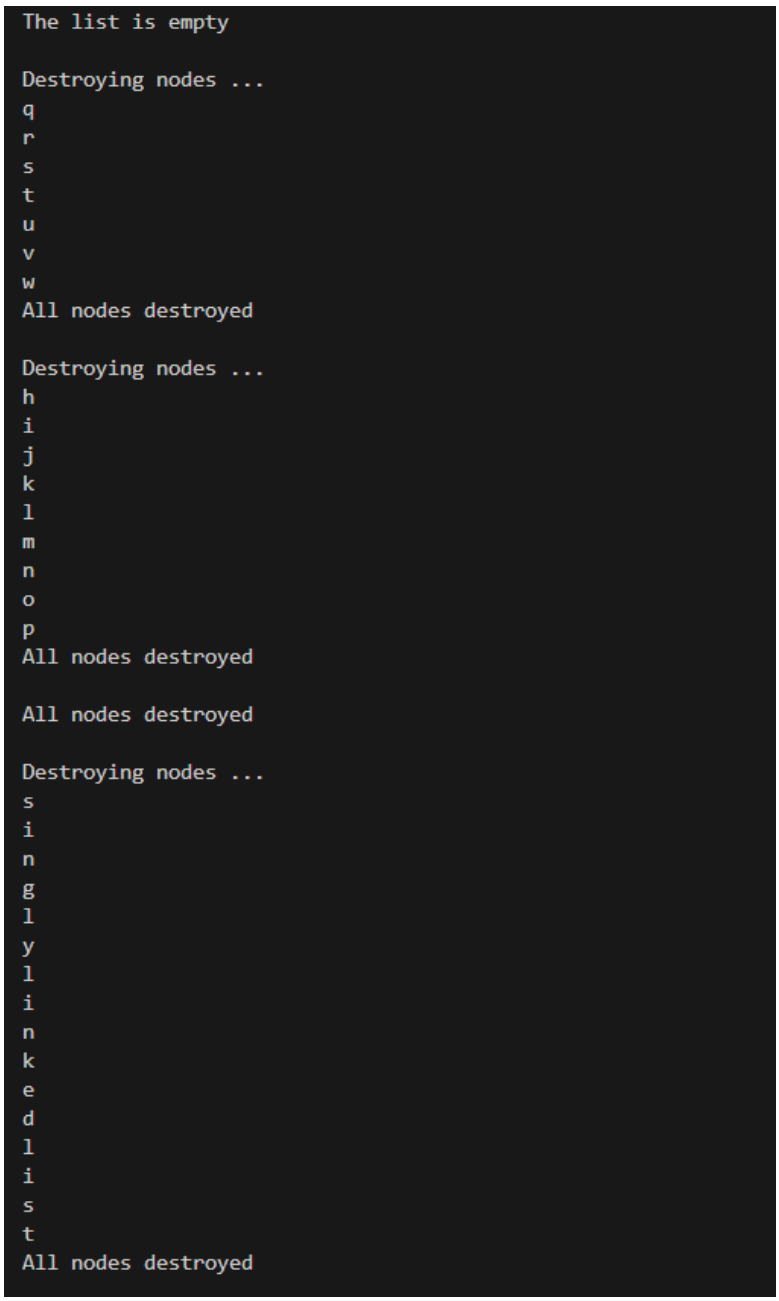
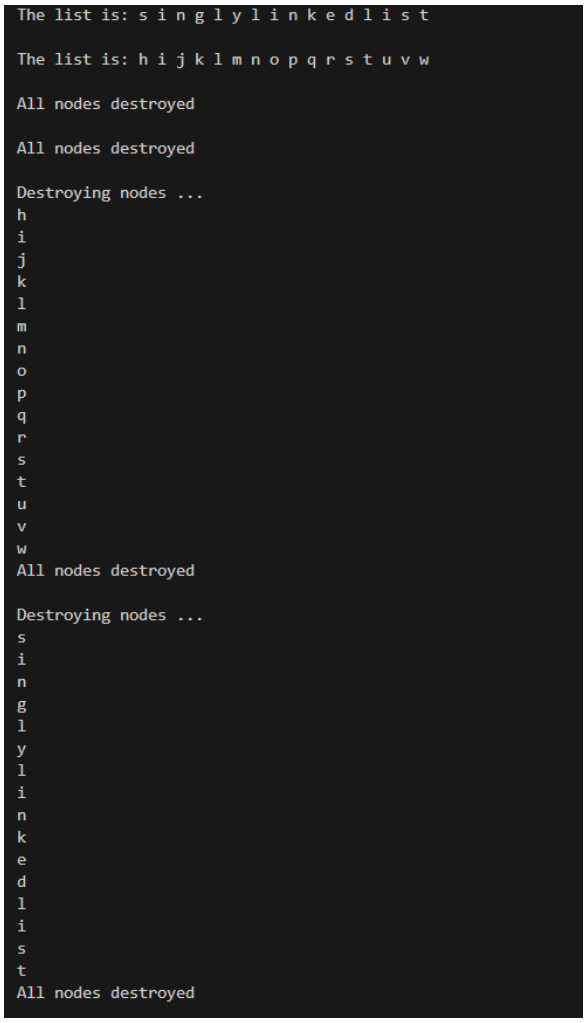


Figure 3: Results of concatenation of a list with itself

The fourth test attempts concatenation with empty lists, in one case the base and in another case the added list. In Figure 4 it can be seen how the lists are filled with only the contents of the non-empty list.



(a) Concatenation of an empty list with a regular list

(b) Concatenation of a list with an empty list

Figure 4: Results of list concatenation involving empty lists

Additionally, the case were both lists are empty was also tested and resulted in an empty list just as expected.

In the last test the lists were filled twice to check if the lists were overridden. In Figure 5 it can be seen that the lists have been increased but no override is present.

Figure 5: Results of filling the list twice

3.3 Questions

Question a

The stack and the queue mirror each other in their arrangement. The working principle behind the former is LIFO, where the latest items are taken out in order to be able to take the first ones put in. The principle behind the latter is FIFO, where the items are accessed in the order they were put in.[3] A linked list is a structure in which each node points to the next one in the line. A node includes their individual data and the address to the other node in the order.[4] On the other hand the tree arranges the nodes in a hierarchical structure. There is a central node, from which the paths start, called the ”root”, and all nodes originating from this are

named ”children” or ”grandchildren” depending on their position in the hierarchy. [1]

Question b

In this section use cases are going to be given for each of the 4 structures. Firstly, any scenario that would require back-tracking would benefit from the LIFO approach of the stack. If a case would require the distribution of a finite number of elements on a first come first serve basis, then the queue would work best[3]. The use case for linked list is dynamic memory management and image processing[2]. The tree comes into play for hierarchical organization such as folder/sub-folder structures.

Question c

The & is used for the address of the variable, while the $*$ is used to obtain the value at a specified address.

Question d

The best approach is to initialize the variable as NULL. This way the address can be distinguished from the initial randomly assigned value.

Question e

The deletion of the nodes occurs from the last list to the first one, due to the structure of the linked lists. All the nodes contain an address to the next one, and as such, the deletion would occur in the reverse order. Just to be certain, the strings were changed, to see if the size of the lists would affect the deletion order. No change was found.

References

[1] Yasin Cakal. *Trees in C++*. Accessed: 2026-03-02. n.d. URL: `https://codeofcode.org/lessons/trees-in-cpp/`.

[2] GeeksforGeeks. *Applications of Linked List Data Structure*. Accessed: 2026-03-02. n.d. URL: `https://www.geeksforgeeks.org/dsa/applications-of-linked-list-data-structure/`.

[3] GeeksforGeeks. *Difference Between Stack and Queue Data Structures*. Accessed: 2026-03-02. 2024. URL: `https://www.geeksforgeeks.org/dsa/difference-between-stack-and-queue-data-structures/`.

[4] Wikipedia contributors. *Linked list*. Accessed: 2026-03-02. 2026. URL: `https://en.wikipedia.org/wiki/Linked_list`.

1 AI statement

During the writing of this assignment AI tools(such as ChatGPT) were only used to identify errors in the code. After which the code was reevaluated and rewritten by the user themselves.